

QA Interim Assessment

Selenium Automation Test Report

Tested By: Premchand Chellisseril Pramod

Date: 29-06-2026

Git Link: [ust-sdet-training/SDET at premchand assessment 29Jun26](https://github.com/premchand/ust-sdet-training/tree/main/SDET%20at%20premchand%20assessment%2029Jun26)

1. Objective

Objective

The objective of this automation project is to perform a UI scenario using Selenium

User Story

As a user, I want to search for stays by destination, dates, guests, and rooms so I can find suitable accommodation for my trip.

2. Flow Under Test

Scenario 1: Navigate to Home Page and search for a hotel

1. Open Booking.com
2. Select Stays
3. Enter destination → e.g., "London"
4. Select check-in and check-out dates
5. Select occupancy:
 - 2 adults
 - 0 children
 - 1 room

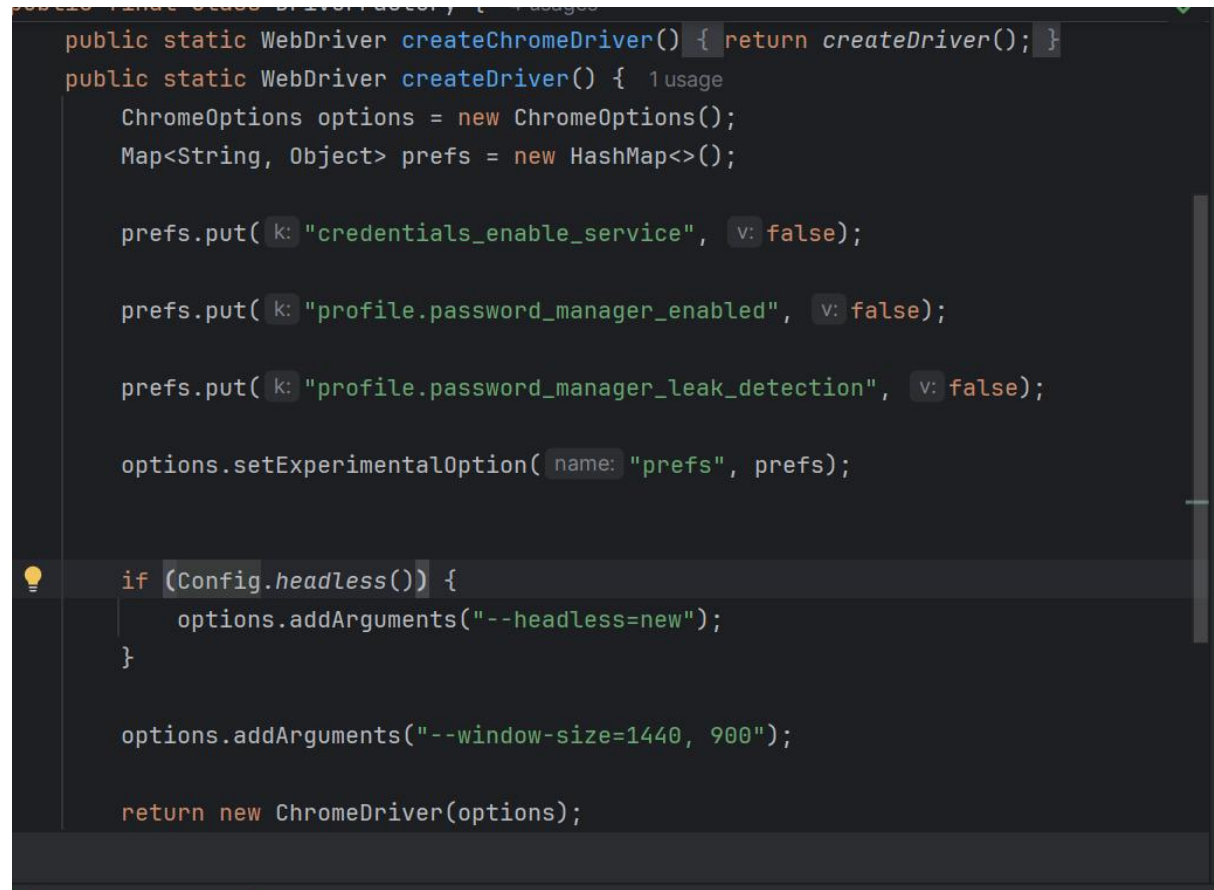
3. Environment & Setup

Configuration

```
public static String baseUrl(){  
    return System.getProperty("baseUrl","https://www.booking.com/").replaceAll("/$", "");  
}
```

```
public static boolean headless(){  
    return Boolean.parseBoolean(System.getProperty("headless","false"));  
}
```

Driver Configuration

A screenshot of a code editor with a dark theme. The code is in Java and defines two static methods for creating a WebDriver. The first method, createChromeDriver, calls createDriver. The second method, createDriver, initializes ChromeOptions and a HashMap of preferences. It sets preferences for 'credentials_enable_service', 'profile.password_manager_enabled', and 'profile.password_manager_leak_detection' to false. It then sets an experimental option 'prefs' to the preferences map. A lightbulb icon indicates a suggestion for an if-statement block that checks Config.headless() and adds the argument '--headless=new' to the options. Finally, it adds the window size argument '--window-size=1440, 900' and returns a new ChromeDriver instance.

```
public static WebDriver createChromeDriver() { return createDriver(); }  
public static WebDriver createDriver() { 1 usage  
    ChromeOptions options = new ChromeOptions();  
    Map<String, Object> prefs = new HashMap<>();  
  
    prefs.put( k: "credentials_enable_service", v: false);  
  
    prefs.put( k: "profile.password_manager_enabled", v: false);  
  
    prefs.put( k: "profile.password_manager_leak_detection", v: false);  
  
    options.setExperimentalOption( name: "prefs", prefs);  
  
    if (Config.headless()) {  
        options.addArguments("--headless=new");  
    }  
  
    options.addArguments("--window-size=1440, 900");  
  
    return new ChromeDriver(options);
```

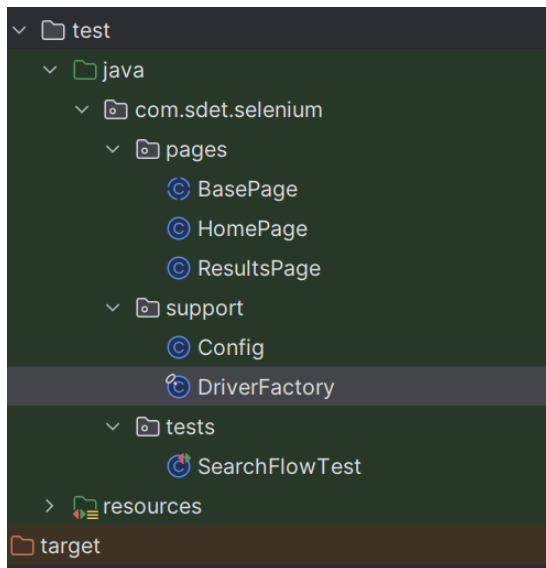
Execution Command:

Mvn test

4. Framework Design

The automation framework follows the Page Object Model (POM) design pattern.

Folder Structure



Components

Tests

Contains SearchFlowTest

Pages

Contains Page Object classes: HomePage, BasePage, ResultsPage

Responsibilities:

- Page locators
- Page actions
- Reusable methods

Support

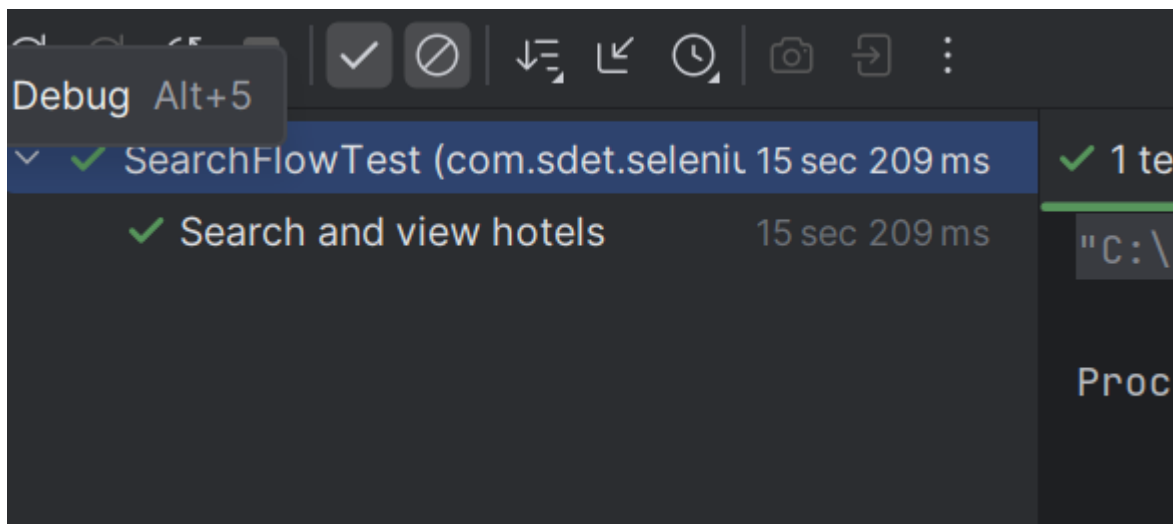
Contains all configurations

5. Test Scenarios Covered

Positive Scenarios

Scenario : Navigate to Home Page and search for a hotel

STATUS: PASSED



7. Execution Result

Metric	Value
--------	-------

Total Tests	1
-------------	---

Passed	1
--------	---

Failed	0
--------	---

Skipped	0
---------	---

8. Conclusion

The Selenium automation framework successfully validates and automates the given UI scenario